

Problem 1:

Answer: 0.0979

Problem 2:

Answer: 40.42

MATLAB Code:

```
%% Problem 2.1
syms x1 x2
f = 100*(x2 - x1^2)^2 + (1 - x1)^2;
varibales = [x1,x2];
f_gradient = gradient(f,varibales);
f_hessian = hessian(f,varibales);

%% Problem 1
syms d
p = 1;D = 1;t =1;
sigma =(1.11 + 1.11*(d/D)^-0.18)/(D - d)*t;
% gradient and hessian of the function
sigma_d = diff(sigma,d);
sigma_dd = diff(sigma_d,d);
%converting symbolic functions to anonymous functions
sigma_function = matlabFunction(sigma);
f1 = matlabFunction(sigma_d);
f2 = matlabFunction(sigma_dd);
% Initial guess for a range of d
initial_guess = 0.1:0.1:5;
critical_points = [];
for i = 1:length(initial_guess)
    if(initial_guess(i) == 1)
        continue;
    end
    solution = fsolve(f1,initial_guess(i));
    if(imag(solution) == 0 && all(abs(critical_points - solution) > 1e-5))
        critical_points = [critical_points,solution];
    end
end
% checking critical points to see if they are minimizers
minimizers = [];
for i = 1:length(critical_points)
    hessian = f2(critical_points(i));
    if (hessian > 0)
        minimizers = [minimizers,critical_points(i)];
    end
end
strict_local_minimizer = min(minimizers);

%Problem 1 with gradient descent approach
initial_guess =0.9;
max_iteration = 1000;
```

```

d = initial_guess;
learning_rate = 0.001;
max_tolerance = 1e-6;
for i =1:max_iteration
    f1_value = sigma_function(d);
    d_new = d - learning_rate*f1(d);
    if (abs(sigma_function(d_new) - f1_value) < max_tolerance && ...
        abs(local_minimizer - d_new)< max_tolerance)
        disp("both approaches return the same value");
        break
    end
    d = d_new;
end

%% problem 2
syms x
V = -(x)*(210 - 2*x)*(297 - 2*x);
V_d = diff(V,x);
V_dd = diff(V_d,x);

%converting symbolic functions to anonymous functions
f1 = matlabFunction(V_d);
f2 = matlabFunction(V_dd);
% Initial guess for a range of d
initial_guess = 0.5:0.5:50;
critical_points = [];
for i = 1:length(initial_guess)
    solution = fsolve(f1,initial_guess(i));
    if(all(abs(critical_points - solution) > 1e-5))
        critical_points = [critical_points,solution];
    end
end
% checking critical points to see if they are minimizers
minimizers = [];
for i = 1:length(critical_points)
    hessian = f2(critical_points(i));
    if (hessian > 0)
        minimizers = [minimizers,critical_points(i)];
    end
end
local_minimizer = min(minimizers);

```

